

GUÍA DE COMPLEJIDADES

TDA y estructuras de datos

Explicación simple de por qué cada operación es constante, lineal, logarítmica o cuadrática

Objetivo: Entender el costo de cada operación y poder justificarlo en el examen, no solo memorizar la tabla.

Material de estudio organizado tema por tema

Índice

1. Cómo leer los costos
2. PilaTDA
3. ColaTDA
4. Cola con prioridad
5. ConjuntoTDA
6. Diccionario simple
7. Diccionario múltiple
8. Lista enlazada simple
9. Árbol Binario de Búsqueda (ABB)
10. AVL
11. Árbol B
12. Grafo con matriz de adyacencia
13. Grafo con lista de adyacencia
14. Cuadro final para memorizar

1. Cómo leer los costos

La complejidad indica cómo aumenta el trabajo de un algoritmo cuando crece la cantidad de elementos. No mide segundos exactos: describe la tendencia del costo.

Notación	Nombre	Idea para recordarla
$C u O(1)$	Constante	Siempre hace una cantidad fija de pasos.
$L u O(n)$	Lineal	Puede recorrer los n elementos una vez.
$O(\log n)$	Logarítmica	En cada paso descarta una gran parte del problema.
$O(n^2)$	Cuadrática	Hay dos recorridos anidados sobre n elementos.
$O(h)$	Depende de la altura	Recorre un camino del árbol desde la raíz.
$O(n + m)$	Vértices más aristas	Recorre todos los vértices y todas las aristas.

Regla clave: Si una operación solo cambia una referencia o un índice, suele ser $O(1)$. Si debe buscar o recorrer elemento por elemento, suele ser $O(n)$.

2. PilaTDA

Una pila trabaja con el criterio LIFO: el último elemento que entra es el primero que sale. Las implementaciones cambian según dónde se ubica el tope.

Operación	PilaTI	PilaTF	PilaLD
InicializarPila	C	C	C
Apilar	L	C	C
Desapilar	L	C	C
Tope	C	C	C
PilaVacía	C	C	C

PilaTI: tope al inicio del arreglo

```
Tope
v
[A][B][C][ ]
```

Apilar es $O(n)$ porque el elemento nuevo debe entrar en la posición 0. Para hacer lugar, todos los elementos existentes se corren una posición hacia la derecha.

```
Antes:  [A][B][C][ ]
Apilar X: [X][A][B][C]
```

Desapilar también es $O(n)$: se elimina la posición 0 y los demás elementos deben correrse hacia la izquierda para que el nuevo tope vuelva a quedar al inicio.

PilaTF: tope al final del arreglo

Apilar y desapilar son $O(1)$ porque se trabaja en la última posición ocupada. Para apilar se escribe en la siguiente posición libre; para desapilar se reduce el índice del tope. No hay corrimientos.

```
Antes:  [A][B][C][ ]
Apilar X: [A][B][C][X]
Desapilar: bajar el índice del tope
```

PilaLD: lista dinámica

Apilar y desapilar son $O(1)$ porque se modifica el comienzo de la lista enlazada. No se mueven datos: solo se cambian referencias.

```
nuevo.sig = primero;
primero = nuevo;      // apilar

primero = primero.sig; // desapilar
```

Para el examen: PilaTI es lineal por los corrimientos. PilaTF y PilaLD son constantes porque trabajan directamente sobre el tope.

3. ColaTDA

Una cola trabaja con el criterio FIFO: el primero que entra es el primero que sale.

Operación	ColaPU	ColaPI	ColaLD
InicializarCola	C	C	C
Acolar	L	C	C
Desacolar	C	L	C
Primero	C	C	C
ColaVacía	C	C	C

ColaPU: último ingresado en la posición inicial

```

Ingresaron A, B y C:
[C][B][A]
  ^   ^
  último primero

```

Acolar es $O(n)$ porque el nuevo elemento debe colocarse en la posición inicial. Para hacerlo, todos los anteriores se desplazan hacia la derecha.

Desacolar es $O(1)$ porque el primero en entrar se encuentra al final de la zona ocupada. Se reduce la cantidad de elementos sin realizar corrimientos.

ColaPI: primer ingresado en la posición inicial

```

[A][B][C]
  ^   ^
  primero último

```

Acolar es $O(1)$ porque el nuevo elemento se agrega al final de la zona ocupada. Desacolar es $O(n)$ porque se elimina el elemento de la posición 0 y hay que correr los restantes hacia la izquierda.

ColaLD: lista dinámica con primero y último

Acolar es $O(1)$ porque se conecta el nodo nuevo después de último. Desacolar es $O(1)$ porque se avanza la referencia primero al nodo siguiente.

```

ultimo.sig = nuevo;
ultimo = nuevo;      // acolar

primero = primero.sig; // desacolar

```

Idea central: En arreglos, la operación realizada en la posición 0 provoca corrimientos. La realizada al final es constante.

4. Cola con prioridad

En una cola con prioridad, cada valor tiene una prioridad. La cola debe mantenerse ordenada para que el elemento con mayor prioridad pueda salir primero.

Operación	ColaPrioridadDA	ColaPrioridadAO	ColaPrioridadLD
InicializarCola	C	C	C
AcolarPrioridad	L	L	L
Desacolar	C	C	C
Primero	C	C	C
ColaVacía	C	C	C
Prioridad	C	C	C

Diferencia entre DA, AO y LD

- ColaPrioridadDA: usa dos arreglos paralelos, uno para los datos y otro para las prioridades.
- ColaPrioridadAO: usa un arreglo de objetos; cada objeto guarda juntos el dato y su prioridad.
- ColaPrioridadLD: usa nodos enlazados; cada nodo contiene dato, prioridad y referencia al siguiente.

Por qué AcolarPrioridad es $O(n)$

El elemento nuevo no puede colocarse simplemente al principio o al final. Primero debe encontrarse la posición correcta comparando su prioridad con las prioridades existentes.

Antes: [A,1] [B,3] [C,5]
 Entra X con prioridad 4
 Después: [A,1] [B,3] [X,4] [C,5]

En los arreglos, además de buscar la posición, puede ser necesario desplazar valores. En la lista dinámica no hay corrimientos, pero igualmente hay que recorrer los nodos hasta encontrar la posición. En ambos casos domina el recorrido $O(n)$.

Importante: $O(2n)$ sigue siendo $O(n)$. Las constantes se ignoran al expresar el orden de complejidad.

5. ConjuntoTDA

Un conjunto almacena elementos sin repetidos. ConjuntoA usa un arreglo y ConjuntoLD una lista dinámica.

Operación	ConjuntoA	ConjuntoLD
InicializarConjunto	C	C
Agregar	L	L
Sacar	L	L
Pertenece	L	L
Elegir	C	C
ConjuntoVacío	C	C

InicializarConjunto - $O(1)$

Solo se asigna cantidad = 0 o primero = null. No se recorren elementos.

Agregar - $O(n)$

Antes de agregar hay que verificar que el elemento no esté repetido. Esa verificación usa Pertenece y puede recorrer todo el conjunto. La inserción posterior puede ser constante, pero $O(n) + O(1) = O(n)$.

Sacar - $O(n)$

Primero se busca el elemento. En un arreglo también puede haber corrimientos; en una lista se recorren nodos y se modifican referencias. La búsqueda hace que el costo sea lineal.

Pertenece - $O(n)$

Se compara el valor buscado con los elementos hasta encontrarlo o terminar el conjunto. El peor caso revisa todos los elementos.

Elegir y ConjuntoVacío - $O(1)$

Elegir devuelve cualquier elemento, por ejemplo el primero. ConjuntoVacío solo revisa cantidad == 0 o primero == null. Ambas operaciones usan acceso directo.

6. Diccionario simple

En un diccionario simple, cada clave tiene un único valor asociado. DicSimpleA usa arreglos y DicSimpleL usa una lista enlazada.

```
DNI 44556677 -> "Nico"
Código 18    -> "Producto A"
```

Operación	DicSimpleA	DicSimpleL
InicializarDiccionario	C	C
Agregar	L	L
Eliminar	L	L
Recuperar	L	L
Claves	L	L

Inicializar - $O(1)$

Solo se deja la estructura vacía mediante una variable o referencia.

Agregar - $O(n)$

Hay que buscar la clave para saber si ya existe. Si existe, se actualiza su valor; si no existe, se agrega una entrada nueva. La búsqueda puede recorrer todo el diccionario.

Eliminar - $O(n)$

Primero se localiza la clave. En arreglo puede haber corrimientos; en lista se ajustan referencias. El recorrido para encontrarla es lineal.

Recuperar - $O(n)$

Para devolver el valor asociado se busca la clave una por una. Sin una estructura de acceso directo, el peor caso revisa n entradas.

Claves - $O(n)$

Para construir el conjunto de claves se visita cada entrada del diccionario una vez.

7. Diccionario múltiple

En un diccionario múltiple, cada clave puede tener un conjunto de valores asociados.

```
Materia -> {Programación, Redes, Matemática}
Alumno  -> {Nico, Ana, Juan}
```

Operación	DicMultipleA	DicMultipleL
InicializarDiccionario	C	C
Agregar	L	L
Eliminar	L	L
EliminarValor	L	L
Recuperar	L	L
Claves	L	L

Agregar - $O(n)$

Primero hay que localizar la clave. También puede verificarse que el valor no esté repetido dentro de su conjunto. Esas búsquedas hacen que el costo sea lineal.

Eliminar - $O(n)$

Eliminar borra una clave completa junto con todos sus valores. La clave debe buscarse antes de quitarla.

EliminarValor - $O(n)$

Primero se busca la clave y después el valor específico dentro de los valores asociados. Esta operación existe en el múltiple porque una clave puede conservar otros valores.

Recuperar y Claves - $O(n)$

Recuperar debe localizar la clave para devolver su conjunto de valores. Claves recorre todas las entradas para generar el conjunto de claves.

Diferencia con el simple: En el diccionario simple una clave tiene un solo valor; en el múltiple puede tener varios, por eso aparece EliminarValor.

8. Lista enlazada simple

Cada nodo guarda un dato y una referencia al nodo siguiente. La lista se recorre desde primero hasta null.

```
primero -> [A|sig] -> [B|sig] -> [C|null]
```

Operación o situación	Costo
Acceder al primer nodo	C
Insertar al inicio	C
Eliminar el primer nodo	C
Recorrer la lista completa	$O(n)$
Buscar un valor	$O(n)$
Insertar al final sin referencia al último	$O(n)$
Insertar al final con referencia al último	C
Mezclar dos listas recorriéndolas una vez	$O(n)$

Operaciones sobre el inicio - $O(1)$

Acceder, insertar o eliminar el primer nodo es constante porque existe la referencia directa primero. Solo se leen o cambian unas pocas referencias.

```
nuevo.sig = primero;
primero = nuevo;      // insertar al inicio

primero = primero.sig; // eliminar el primero
```

Recorrer y buscar - $O(n)$

Una lista enlazada no permite saltar directamente al nodo i . Hay que avanzar nodo por nodo. Recorrerla completa o buscar un valor puede visitar los n nodos.

Insertar al final

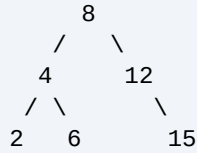
Sin una referencia último, hay que recorrer toda la lista hasta encontrar el nodo cuyo siguiente es null: $O(n)$. Con la referencia último, se conecta el nuevo nodo directamente: $O(1)$.

Mezclar dos listas - $O(n)$

Si cada nodo de ambas listas se visita una sola vez, el costo es $O(n + m)$. Considerando n como la cantidad total de nodos recorridos, se expresa $O(n)$.

9. Árbol Binario de Búsqueda (ABB)

En un ABB, los valores menores quedan en el subárbol izquierdo y los mayores en el derecho.



Operación	Costo
InicializarÁrbol	C
ÁrbolVacio	C
Raíz	C
HijoIzq	C
HijoDer	C
AgregarElem	O(h)
EliminarElem	O(h)
Buscar / existe elemento	O(h)
Mayor / menor	O(h)
Profundidad de un elemento	O(h)
Preorden / inorden / postorden	O(n)
Contar nodos	O(n)
Calcular altura	O(n)

Operaciones de acceso directo - O(1)

Inicializar, comprobar si está vacío, leer la raíz y acceder a sus hijos usan referencias directas. No recorren el árbol.

Operaciones que siguen un camino - O(h)

Buscar, agregar, eliminar, encontrar el mayor o menor y calcular la profundidad recorren un solo camino desde la raíz. La cantidad de pasos depende de h, la altura del árbol.

```

Buscar 6:
8 -> izquierda
4 -> derecha
6 -> encontrado
  
```

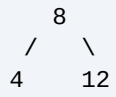
Operaciones que visitan todo el árbol - O(n)

Los recorridos, contar nodos y calcular la altura necesitan visitar todos los nodos. Por eso dependen de n, la cantidad total de nodos.

Balanceado frente a degenerado

Si el ABB está balanceado, h se aproxima a $\log n$ y buscar cuesta $O(\log n)$. Si se degenera como una lista, h puede llegar a n y buscar cuesta $O(n)$.

Balanceado:



Degenerado:



10. AVL

Un AVL es un ABB que se mantiene balanceado automáticamente. En cada nodo, la diferencia entre las alturas de los subárboles izquierdo y derecho es como máximo 1.

Operación	Costo
Buscar	$O(\log n)$
Insertar	$O(\log n)$
Eliminar	$O(\log n)$
Rotación simple	C
Rotación doble	C
Recorrido completo	$O(n)$

Buscar - $O(\log n)$

El árbol permanece balanceado, por lo que su altura es aproximadamente $\log n$. La búsqueda recorre un solo camino de esa altura.

Insertar y eliminar - $O(\log n)$

Primero se recorre el camino para encontrar la posición o el elemento. Después se revisa el balance al volver hacia la raíz. Todo el proceso está limitado por la altura logarítmica.

Rotaciones - $O(1)$

Una rotación reorganiza una cantidad fija de nodos y referencias. Una rotación doble son dos rotaciones simples; $O(1) + O(1)$ sigue siendo $O(1)$.

Antes:	Después:
<pre> 30 / 20 / 10 </pre>	<pre> 20 / \ 10 30 </pre>

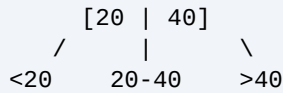
Recorrido completo - $O(n)$

Aunque el árbol esté balanceado, un recorrido completo debe visitar los n nodos.

Diferencia con ABB: El ABB común puede degenerarse y llegar a $O(n)$. El AVL conserva $O(\log n)$ para buscar, insertar y eliminar.

11. Árbol B

Un árbol B es un árbol de búsqueda balanceado en el que cada nodo puede almacenar varias claves y tener varios hijos. Todas las hojas se encuentran al mismo nivel.



Operación	Costo conceptual
Buscar	$O(\log n)$
Insertar	$O(\log n)$
Eliminar	$O(\log n)$
Split de nodo	C respecto de la altura
Préstamo	C respecto de la altura
Fusión	C respecto de la altura

Buscar - $O(\log n)$

En cada nodo se comparan varias claves y se selecciona un hijo. Como el árbol está balanceado y tiene mucha ramificación, se recorren pocos niveles.

Insertar - $O(\log n)$

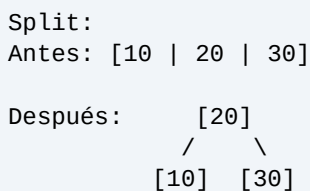
Se baja hasta la hoja correspondiente. Si el nodo está lleno, se divide mediante un split y la clave central sube al padre. La corrección puede propagarse hacia arriba, pero recorre como máximo la altura.

Eliminar - $O(\log n)$

Después de buscar la clave, una falta de claves puede corregirse mediante préstamo o fusión. Las correcciones pueden subir hasta la raíz, por lo que el costo total depende de la altura logarítmica.

Split, préstamo y fusión - constantes respecto de la altura

Estas operaciones actúan localmente sobre uno o pocos nodos. Si el orden máximo del árbol se considera fijo, mueven una cantidad acotada de claves y se toman como $O(1)$ respecto de la altura. Si el orden fuera variable, su costo dependería de ese orden.



12. Grafo con matriz de adyacencia

Una matriz de adyacencia tiene una fila y una columna por vértice. Cada celda indica si existe una arista o guarda su peso.

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

Operación	Costo orientativo
InicializarGrafo	$O(n^2)$
AgregarVertice	$O(n)$
EliminarVertice	$O(n)$
Vertices	$O(n)$
AgregarArista	$O(n)$ por búsqueda de índices
EliminarArista	$O(n)$ por búsqueda de índices
ExisteArista	$O(n)$ por búsqueda de índices
PesoArista	$O(n)$ por búsqueda de índices
Recorrer todas las posibles aristas	$O(n^2)$

InicializarGrafo - $O(n^2)$

La matriz tiene n filas y n columnas. Inicializar todas sus celdas requiere $n \times n$ operaciones.

Agregar y eliminar vértice - $O(n)$

Agregar un vértice implica inicializar una nueva fila y una nueva columna. Eliminarlo implica limpiar sus conexiones. Según la implementación de la materia, se considera lineal.

Operaciones sobre aristas - $O(n)$

Si ya se conocieran los índices i y j , modificar o consultar `matriz[i][j]` sería $O(1)$. Sin embargo, el TDA recibe los valores de los vértices y primero debe buscarlos en la colección de vértices. Esa búsqueda es $O(n)$ y domina el costo.

```

buscar índice de A ->  $O(n)$ 
buscar índice de C ->  $O(n)$ 
matriz[i][j] = 1    ->  $O(1)$ 
Total:  $O(n)$ 

```

Recorrer todas las aristas posibles - $O(n^2)$

Se revisan todas las celdas mediante dos recorridos anidados: una vez por cada fila y cada columna.

Memoria: La matriz reserva espacio para n^2 conexiones, incluso cuando existen pocas aristas.

13. Grafo con lista de adyacencia

Cada vértice guarda una lista con los vértices a los que está conectado. Solo se almacenan las aristas que realmente existen.

```
A -> B -> C
B -> C
C -> A
```

Operación	Costo orientativo
InicializarGrafo	C
AgregarVertice	C
EliminarVertice	$O(n + m)$
Vertices	$O(n)$
AgregarArista	$O(n)$ por buscar vértices
EliminarArista	$O(n + \text{grado}(v))$
ExisteArista	$O(n + \text{grado}(v))$
PesoArista	$O(n + \text{grado}(v))$
Recorrer todo el grafo	$O(n + m)$

Aquí n es la cantidad de vértices, m es la cantidad total de aristas y $\text{grado}(v)$ es la cantidad de aristas salientes del vértice consultado.

Inicializar y agregar vértice - $O(1)$

Inicializar asigna una referencia vacía. Agregar un vértice al inicio de la lista principal cambia pocas referencias. Esto supone que no se verifica previamente si está repetido.

Eliminar vértice - $O(n + m)$

Hay que encontrar el vértice entre los n existentes y eliminar todas las aristas que salen de él o llegan a él. Para retirar las referencias entrantes pueden recorrerse todas las listas, que en conjunto contienen m aristas.

Agregar arista - $O(n)$

Primero se buscan los vértices de origen y destino. Una vez localizado el origen, insertar una arista al principio de su lista es $O(1)$. La búsqueda de vértices domina.

Eliminar, comprobar o leer una arista - $O(n + \text{grado}(v))$

Se busca el vértice de origen entre n vértices y después se recorre su lista de adyacencia, que contiene $\text{grado}(v)$ aristas.

Recorrer todo el grafo - $O(n + m)$

Se visita cada vértice una vez y cada arista almacenada una vez. A diferencia de la matriz, no se revisan conexiones inexistentes.

Cuándo conviene: La lista de adyacencia suele ser más eficiente en memoria para grafos dispersos, es decir, con pocas aristas respecto de n^2 .

14. Cuadro final para memorizar

Estructura	Operaciones rápidas	Operaciones que recorren
PilaTI	Tope, vacía	Apilar y desapilar por corrimientos
PilaTF	Apilar, desapilar, tope	-
PilaLD	Apilar y desapilar al inicio	Buscar/recorrer si se necesitara
ColaPU	Desacolar	Acolar por corrimientos
ColaPI	Acolar	Desacolar por corrimientos
ColaLD	Acolar con último y desacolar con primero	-
Cola prioridad	Desacolar/primero si ya está ordenada	Acolar para encontrar la posición
Conjunto	Elegir, vacío	Pertenece, agregar, sacar
Diccionarios	Inicializar	Buscar clave, recuperar, eliminar, claves
Lista enlazada	Inicio y final con referencia último	Buscar y llegar al final sin último
ABB	Accesos directos	Un camino $O(h)$ o todo el árbol $O(n)$
AVL	Rotaciones $O(1)$	Buscar/insertar/eliminar $O(\log n)$, recorrido $O(n)$
Árbol B	Operaciones locales	Buscar/insertar/eliminar $O(\log n)$
Matriz de adyacencia	Celda $O(1)$ si se conocen índices	Matriz completa $O(n^2)$; buscar índices $O(n)$
Lista de adyacencia	Agregar vértice $O(1)$	Todo el grafo $O(n+m)$

Reglas rápidas de razonamiento

- Si se cambia solo una referencia, un índice o una celda conocida: $O(1)$.
- Si hay que buscar un elemento sin acceso directo: $O(n)$.
- Si hay que desplazar todos los elementos de un arreglo: $O(n)$.
- Si un árbol balanceado reduce el problema por niveles: $O(\log n)$.
- Si se visita cada nodo del árbol: $O(n)$.
- Si se recorren todas las celdas de una matriz $n \times n$: $O(n^2)$.
- Si se recorren todos los vértices y todas las aristas existentes: $O(n + m)$.

Método para justificar: Decí qué debe recorrer o modificar la operación. Después relacioná esa cantidad con n , h , m o $\text{grado}(v)$. Esa explicación vale más que repetir la letra de memoria.

Glosario mínimo

Símbolo	Significado
n	Cantidad de elementos o vértices.
m	Cantidad de aristas.
h	Altura del árbol.
grado(v)	Cantidad de aristas asociadas al vértice v.
C	Constante, $O(1)$.
L	Lineal, $O(n)$.